

Rapport de Projet d'optimisation Opticalflow

Anas Amrouche

12 avril 2026

Résumé

Le problème du flux optique consiste à déterminer les mouvements dans un flux vidéo.

Le résoudre est essentiel dans les tâches de détection de mouvements, de reconnaissance de formes et de vision par ordinateur de façon plus générale.

Table des matières

1	Introduction	3
2	Étude du modèle	4
2.1	Caractère propre de la fonctionnelle d'énergie	4
2.2	Stricte convexité de la fonctionnelle d'énergie	5
3	Approximations numériques	5
3.1	Gauss-Seidel	6
3.1.1	Version multi-échelle	6
3.2	Jacobi	6
3.3	Descente de Gradient	6
4	Implémentation informatique	6
5	Analyse des résultats	7
5.1	Vitesse de convergence	8
5.2	Influence de α	8
5.3	Détection des faibles détails	8
5.4	Grands mouvements	8
5.5	Norme L^1	9
5.6	Benchmarks	9
6	Développements futurs au delà du cadre de l'UE	9
6.1	Gauss-Seidel rouge-noir	9
6.2	Lucas-Kanade	9
6.3	Étape de convolution	9

6.4	Rendu en temps réel	9
6.5	Prise en charge des couleurs	10
6.6	Réorganisation du module	10
7	Lien vers le dépôt et les vidéos générées	10

1 Introduction

Pour le résoudre nous fixons comme hypothèse préliminaire que la luminosité d'un pixel se déplaçant sur la vidéo est constante. Il est décrit par deux grandeurs scalaires, u représentant le déplacement horizontal et v le déplacement vertical. Partant de cette hypothèse on obtient :

$$I(x + u, y + v, t + 1) = I(x, y, t) + \frac{\partial I}{\partial x}(x, y, t) \cdot u + \frac{\partial I}{\partial y}(x, y, t) \cdot v + \frac{\partial I}{\partial t}(x, y, t) = I(x, y, t) \quad (1)$$

$$\iff \frac{\partial I}{\partial x}(x, y, t) \cdot u + \frac{\partial I}{\partial y}(x, y, t) \cdot v + \frac{\partial I}{\partial t} = 0 \quad (2)$$

Par hypothèse de constance de la luminosité.

L'espace des solutions en un point est un espace affine de dimension 1. Il existe une infinité de solutions. C'est le problème de l'ouverture (aperture en anglais). Les méthodes de résolution ont toutes pour principe d'introduire une contrainte pour réduire l'espace des solutions à un point.

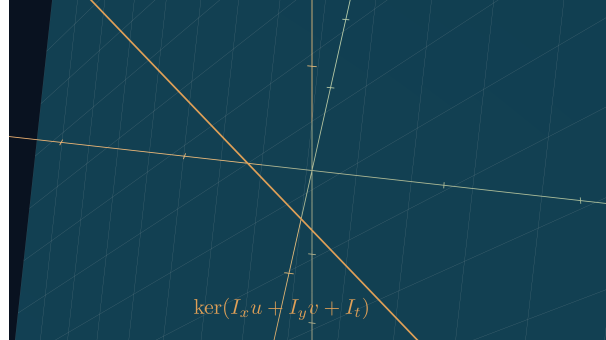


FIGURE 1 – Le sous-espace affine de dimension 1 vérifiant l'équation en un pixel

La méthode de Horn-Schunck passe par une contrainte de régularisation du flux. On va supposer que le flot a une régularité qui dépendra d'un paramètre α . La fonctionnelle E de Horn-Schunck sera la fonction à optimiser.

$$E(u, v) = \int_{\Sigma} (I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2) \quad (3)$$

avec I_k la dérivée selon le paramètre k , et Σ la surface de l'image.

Il existe une version exploitant la norme L^1 pour préserver les discontinuités :

$$E(u, v) = \int_{\Sigma} (I_x u + I_y v + I_t)^2 + \alpha^2 (|\nabla u| + |\nabla v|) \quad (4)$$

Le deuxième terme de l'intégrande correspond à une pénalisation et permet de transformer un problème d'algèbre linéaire en un problème d'optimisation d'une fonctionnelle. La clé sera de trouver un compromis entre la minimisation du terme 1 lié à l'exactitude du flux et l'amplitude de ses variations spatiales liées au terme 2. Ce minimiseur existe et dépend continûment du paramètre α (E étant continu en α).

Ce projet a pour but de mettre en place informatiquement les méthodes de descente de gradient et de résolution par Gauss-Seidel des équations d'Euler-Lagrange. Il se fera dans un langage assez performant (en l'occurrence Rust) pour accélérer le traitement de l'image. Pour faciliter la génération de détections, le code sera exporté en un module Python. 4 vidéos différentes seront traitées par chaque méthode avec des paramètres différents. L'approche par Gauss-Seidel se fera de façon simple puis multi-échelle avec la norme L^2 . Celle par descente de gradient se fera de façon simple mais traitera le cas des normes L^1 et L^2 .

2 Étude du modèle

Dans le cadre d'une implémentation numérique, le problème est discrétisé. Dans la suite nous définirons chaque frame de l'image comme une matrice $\mathcal{M}_{H \times W}(\mathbb{R})$

La contrainte de Horn-Schunck s'écrit alors :

$$E_{L^2}(u, v) = \sum_{i,j} \underbrace{(I_x[i, j] \cdot u[i, j] + I_y[i, j] \cdot v[i, j] + I_t[i, j])^2}_{\text{Terme 1}} + \underbrace{\alpha^2 (\|\nabla u[i, j]\|^2 + \|\nabla v[i, j]\|^2)}_{\text{Terme 2}}$$

$$E_{L^1}(u, v) = \sum_{i,j} \underbrace{(I_x[i, j] \cdot u[i, j] + I_y[i, j] \cdot v[i, j] + I_t[i, j])^2}_{\text{Terme 1}} + \underbrace{\alpha^2 (|\nabla u[i, j]| + |\nabla v[i, j]|)}_{\text{Terme 2}}$$

note : l'emploi non standard des crochets met l'accent sur la discrétisation du problème et sa résolution informatique. L'objectif de cette section sera de montrer la stricte convexité de la fonction ainsi que son caractère propre pour pouvoir conclure à l'unicité d'un minimiseur.

2.1 Caractère propre de la fonctionnelle d'énergie

Les deux termes rentrant en jeu dans la somme sont positifs. Sans perte de généralité concentrons-nous d'abord seulement sur le champ scalaire u . L'objectif va être de montrer que quelle que soit la façon dont on fait tendre la norme du vecteur représentant le champ vers l'infini, la fonctionnelle tendra vers l'infini. En faisant varier seulement l'une des coordonnées du vecteur u , alors $\|\nabla u\| \rightarrow \infty$. Supposons qu'à présent nous faisons diverger $\|u\|$ vers l'infini de façon à ce que $|u_n[i, j] - u_n[i \pm 1, j]| \sim^{n \rightarrow \infty} c \in \mathbb{R}$ c'est-à-dire que la suite u_n est équivalente à une translation. Cette approche est la seule prévenant l'explosion du gradient. Alors sous l'hypothèse qu'il y a du mouvement sur au moins un pixel de l'image, $\exists i, j, (I_x[i, j] \cdot u[i, j] \neq 0 \wedge I_y[i, j] \cdot v[i, j] \neq 0)$ ce qui nous permet d'affirmer de nouveau que la fonctionnelle tend vers l'infini lorsque la norme tend vers l'infini. Elle est minorée par 0 et continue, elle admet donc un minimum global.

2.2 Stricte convexité de la fonctionnelle d'énergie

Elle est somme de la norme élevée au carré d'un terme affine et de la norme au carré du gradient qui est un opérateur linéaire. Elle est donc strictement convexe en ses deux termes donc strictement convexe.

On peut donc conclure que tout minimum local est global.

3 Approximations numériques

Les équations d'Euler-Lagrange nous fournissent les relation :

$$\begin{aligned} I_x(I_x u + I_y v + I_t) - \alpha \Delta u &= 0 \\ I_y(I_x u + I_y v + I_t) - \alpha \Delta v &= 0 \end{aligned}$$

La discrétisation du problème impose quelques définitions liées aux approximations :

$$\begin{aligned} I_x(x, y) &= \begin{cases} \frac{I(x+1, y) - I(x-1, y)}{2} & \text{si } 0 < x < W \\ I(W, y) - I(W-1, y) & \text{si } x = W \\ I(1, y) - I(0, y) & \text{si } x = 0 \end{cases} \\ I_y(x, y) &= \begin{cases} \frac{I(x, y+1) - I(x, y-1)}{2} & \text{si } 0 < y < H \\ I(x, H) - I(x, H-1) & \text{si } y = H \\ I(x, 1) - I(x, 0) & \text{si } y = 0 \end{cases} \\ \Delta u(x, y) &= \bar{u}(x, y) - u(x, y) \end{aligned}$$

$$\begin{aligned} \bar{u}[i, j] &= u[i, j] + \frac{1}{6} \cdot (u[i-1, j] + u[i+1, j] + u[i, j-1] + u[i, j+1]) \\ &+ \frac{1}{12} \cdot (u[i+1, j+1] + u[i+1, j-1] + u[i-1, j+1] + u[i-1, j-1]) \end{aligned} \quad (5)$$

$$u^{k+1} = \bar{u}^k - \frac{I_x(I_x \bar{u}^k + I_y \bar{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2} \quad (6)$$

$$(7)$$

Le noyau de convolution du laplacien est celui dont les poids sont égaux à 1/6 sur les pixels à proximité directe et 1/12 sur ceux du coin.

Lorsque le noyau de convolution du laplacien "dépassé" de l'image, on remplace les pixels morts en dehors de l'image par la valeur du pixel au centre du noyau (condition de Neumann nulle).

$$\begin{aligned} I_x(I_x u + I_y v + I_t) - \alpha^2(\bar{u} - u) &= 0 \\ I_y(I_x u + I_y v + I_t) - \alpha^2(\bar{v} - v) &= 0 \end{aligned}$$

Ce qui nous amène au système :

$$\begin{pmatrix} \alpha^2 + I_x^2 & I_x I_y \\ I_x I_y & \alpha^2 + I_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \alpha^2 \bar{u} - I_x I_t \\ \alpha^2 \bar{v} - I_y I_t \end{pmatrix}$$

Qui nous amènent aux formules :

$$u_{i,j}^{k+1} = \bar{u}_{i,j}^k - I_{x_{i,j}} \frac{I_{x_{i,j}} \bar{u}_{i,j}^k + I_{y_{i,j}} \bar{v}_{i,j}^k + I_{t_{i,j}}}{\alpha^2 + I_{x_{i,j}}^2 + I_{y_{i,j}}^2}$$

$$v_{i,j}^{k+1} = \bar{v}_{i,j}^k - I_{y_{i,j}} \frac{I_{x_{i,j}} \bar{u}_{i,j}^k + I_{y_{i,j}} \bar{v}_{i,j}^k + I_{t_{i,j}}}{\alpha^2 + I_{x_{i,j}}^2 + I_{y_{i,j}}^2}$$

Les méthodes de Gauss-Seidel et Jacobi différeront par le calcul de $\bar{u}$ et $\bar{v}$.

3.1 Gauss-Seidel

Cette méthode utilise les valeurs des pixels précédents actualisés durant l'itération pour le calcul du laplacien. Sa convergence est théoriquement plus rapide que celle de la méthode de Jacobi. Cependant difficilement parallélisable.

3.1.1 Version multi-échelle

Pour gérer les déplacements importants ou réduire les temps de calcul, une pyramide d'image peut-être mise en place. La résolution du problème sur des images réduites puis leur usage pour des images de plus en plus détaillées permet de réduire le temps de calcul et de suivre les pixels se déplaçant très rapidement.

3.2 Jacobi

Bien que théoriquement plus lente, cette méthode a le bénéfice de pouvoir être parallélisée sur un GPU assez facilement car les laplaciens ne dépendront pas d'actualisations qui surviennent sur la même itération de la boucle for.

3.3 Descente de Gradient

Même dans sa version la plus simple (pas fixe) nous verrons que l'algorithme produira un résultat satisfaisant en très peu d'itérations.

4 Implémentation informatique

J'ai codé le module `horn_schunck_rs` pour accélérer les calculs. Celui-ci va tirer parti des crates `numpy_rs` et `Py03`. Chronologiquement j'ai mis en place la méthode de Gauss-Seidel en définissant d'abord une fonction `gauss_seidel` dont le rôle est de traiter les images individuellement. Elle prend en paramètres l'image actuelle pour le calcul des dérivées spatiales et celle qui suit pour celui des dérivées temporelles. Celle-ci est exploitée par `solve_gauss_seidel` qui est la fonction exposée à Python. Elle appellera la précédente sur chaque paire d'images successives et stockera les champs scalaires u et v dans un

array. La méthode du gradient est construite sur le même principe, une fonction encapsulant la logique image par image et une autre l'appelant à chaque frame. Gauss-Seidel ne minimise la fonctionnelle que pour la norme L^2 contrairement à la descente de gradient. J'ai mis en place une version multi-échelle de Gauss-Seidel pour suivre les grands déplacements. Celle-ci downscale chaque frame d'un tiers sur chaque dimension par l'emploi d'un noyau de convolution Gaussien pour éviter l'aliasing (filtre passe-bas empêchant les hautes fréquences de générer des contours prononcés).

$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$
$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{8}$
$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$

Noyau de convolution Gaussien

La méthode du gradient prend en charge le problème sous ses versions norme L^1 et L^2 . Le gradient de la norme L^1 correspond à

$$\nabla E_{u_{i,j}} = 2I_x \cdot (I_x \cdot u_{i,j} + I_y \cdot v_{i,j} + I_t) - \alpha^2 \cdot (\text{sign}(u_{i+1,j} - u_{i,j}) + \text{sign}(u_{i,j} - u_{i-1,j}) + \text{sign}(u_{i,j+1} - u_{i,j}) + \text{sign}(u_{i,j} - u_{i,j-1}))$$

Sous sa forme L^2 :

$$\nabla E_{u_{i,j}} = 2I_x \cdot (I_x \cdot u_{i,j} + I_y \cdot v_{i,j} + I_t - \alpha^2 \cdot \Delta u_{i,j})$$

J'ai implémenté un critère d'arrêt à partir qui met fin aux algorithmes lorsque la diminution devient inférieure à un certain seuil mais j'ai pu constater que le temps de calcul de la fonctionnelle actualisée à chaque étape était supérieure à celui économisé par les arrêts précoces.

5 Analyse des résultats

J'ai choisi quatre vidéos différentes pour pouvoir comparer les méthodes entre elles ainsi que les différents paramètres à leur fournir. La première appelée "falling ball" représente la chute de balles et constitue un test basique de détection de mouvements. La deuxième appelée "bus fight" me permet d'évaluer la convergence des algorithmes dans un contexte chaotique (mouvements brusques de caméra, personnes se déplaçant dans tous les sens, qualité vidéo très pauvre). La troisième appelée "vapeur" m'est venue de la suggestion en fin de présentation du projet. Elle m'a permis de vérifier si les algorithmes permettent de percevoir des détails extrêmement fins liés à la vapeur d'eau. Enfin j'ai pris une vidéo que j'ai sobrement nommée "TGV" montrant un train à haute vitesse se déplaçant pour tester l'intérêt de la méthode pyramidale. À chaque vidéo j'ai généré grâce à un script Python un rendu en noir et blanc dans lequel chaque pixel s'allumait lorsque la norme du vecteur (u, v) dépassait un certain seuil.

5.1 Vitesse de convergence

Je n'ai pas constaté de différence entre 10 et 50 itérations pour la méthode de Gauss-Seidel. En ce qui concerne la descente de gradient, bien choisir les paramètres à l'avance assure aussi une convergence très rapide en 10 itérations, cependant lorsque le pas est trop grand, plus d'itérations deviennent nécessaires pour assurer la convergence. Concernant bus fight j'ai pensé que le caractère instable de la vidéo rendrait la convergence difficile mais ce ne fut pas le cas, la convergence s'est faite à la même vitesse.

5.2 Influence de α

Le résultat était plus ou moins bruité en fonction du paramètre α . Lorsque la vidéo présente peu de mouvements un paramètre α faible génère une détection sensible mais beaucoup de bruit tandis qu'un paramètre élevé permet de "lisser" les flux par la contrainte sur les gradients mais a tendance à "étouffer" le signal. Dans le cas de grands déplacements comme dans TGV, le flux est élevé à certains points de la vidéo. Un paramètre α faible permet d'isoler l'objet en mouvement. Lorsqu'il est élevé, il "tire" vers le haut la norme des flux alentours et l'algorithme est obligé de détecter des mouvements imperceptibles pour réduire les gradients. En somme, c'est l'effet inverse de ce qui est observé dans le cas de faibles mouvements. Dans le cas de falling ball, la vidéo ne comporte que de faibles déplacements. On peut constater au départ que l'algorithme détecte les contours des objets en arrière-plan de la vidéo jusqu'au lâcher de balle. Elles disparaissent ensuite pour pouvoir optimiser la détection du mouvements des ballons sans trop augmenter le gradient. C'est en observant ce phénomène que j'ai pu réaliser la différence fondamentale entre Lucas-Kanade et Horn-Schunck. Lucas-Kanade est une méthode d'optimisation locale. Chaque flux est calculé pour optimiser une fonctionnelle locale indépendamment des flux précédents. Horn-Schunck est une méthode globale. Les grands déplacements affectent les pixels même éloignés comme décrit précédemment. Cette propriété s'illustre encore dans le cas de bus fight où notre Mohammed Ali causait des perturbations à la détection à chaque frappe lorsque α était important.

5.3 Détection des faibles détails

Vapeur a été assez décevante. La vapeur n'a pas été détectée peu importe la paramètre α choisi. Il semblerait qu'il faudrait probablement passer par des méthodes pour augmenter le contraste de la vapeur (peut-être une convolution avec un noyau bien choisi ou traiter la vidéo par un CNN préalablement).

5.4 Grands mouvements

La méthode multi-échelle arrive à suivre correctement le mouvement du train, cependant elle produit beaucoup d'artefacts à l'image. Plus la hauteur de la pyramide est élevée, plus les artefacts se multiplient. Son usage semble intéressant uniquement si le déplacement est très important (type paysage qui défile rapidement) ou bien que la profondeur de récursion est faible.

5.5 Norme L^1

Les discontinuités sont plus marquées comme le montre la foule et le bus dans bus fight qui sont plus nettement dessinés ou bien falling ball où les ballons sont très nets. Cependant la descente de gradient a plus de mal à converger lorsque les pas sont trop grands.

5.6 Benchmarks

Il sert à comparer les vitesses de Gauss-Seidel simple échelle, la descente de gradient et Lucas-Kanade. En moyennant sur 15 itérations pour chaque méthode exécutée sur falling ball avec la valeur 10 comme paramètre d'itérations de chaque méthode, nous obtenons comme résultats :

Méthode (Implémentation)	Moyenne	Écart-type
Lucas-Kanade (OpenCV - C++)	0.281	0.0045
Descente de gradient (Rust)	10.31	0.0980
Gauss-Seidel (Rust)	24.03	0.1426

TABLE 1 – Temps d'exécution sur 15 itérations pour le traitement de falling ball.

La différence de vitesse entre la descente de gradient et Gauss-Seidel est probablement due au calcul d'une division qui est plus complexe à réaliser pour un CPU. La méthode de Lucas-Kanade est extrêmement rapide. L'implémentation en Rust nous permet d'écarter l'hypothèse de lenteur du côté de Python. Le modèle de Horn-Schunck est peu performant en termes de vitesse.

6 Développements futurs au delà du cadre de l'UE

6.1 Gauss-Seidel rouge-noir

Cette méthode accélérerait les calculs en séparant les pixels en deux groupes traités alternativement. Leur répartition se ferait comme celles de cases d'un échiquier. Cela permettrait de paralléliser les calculs par l'emploi de multithreading sur le CPU ou envoi vers le GPU.

6.2 Lucas-Kanade

L'intégrer dans le module serait intéressant.

6.3 Étape de convolution

Explorer si certains noyaux permettraient de mettre en valeur des détails spécifiques.

6.4 Rendu en temps réel

J'ai toujours envie d'essayer de mettre en place une détection en temps réel.

6.5 Prise en charge des couleurs

Pour l'instant une conversion vers des niveaux de gris est réalisée avant le traitement de l'image. J'aimerais explorer si d'autres méthodes ne prenant pas seulement comme hypothèse la constance de luminosité mais aussi de couleur existe. J'aimerais aussi que lors du rendu vidéo, la couleur à la direction de celui-ci.

6.6 Réorganisation du module

Le code est encore en ébauche et d'un point de vue extérieur peu clair. La coexistence de deux groupes de fonction liées à la méthode de Gauss-Seidel (l'un pour celle à simple échelle et l'autre pour l'approche multi-échelle) devrait disparaître. La méthode de descente de gradient est plus élégante car encapsule les approches L^1 et L^2 sans doublons. La distinction entre le module utilities et le module principal est parfois floue.

7 Lien vers le dépôt et les vidéos générées

[Lien vers le dépôt GitHub.](#)

[Pour voir les vidéos générées sans avoir à les calculer soi-même.](#)